

การวิเคราะห์ผล การประเมินประสิทธิภาพเชิงปริมาณ การรู้จำตัวอักษร (OCR) วัด Character Error Rate (CER) ด้วย Levenshtein distance ระหว่างผลลัพธ์ของโมเดลกับข้อความอ้างอิง ดังสูตร
$$“CER” = (S + D + I) / N$$
 โดยที่ \$S\$, \$D\$, \$I\$ คือจำนวนตัวอักษรที่แทนที่ผิด ขาดหายไป และแทรกเกินตามลำดับ และ \$N\$ คือจำนวนตัวอักษรทั้งหมดในข้อความอ้างอิง วัด Levenshtein distance และใช้ชุดทดสอบ ThaiOCRBench ทั้งหมด 2,808 ภาพ จาก 13 ชุด และ thai-ocr-evaluation ทั้งหมด 104 ภาพ ร่วมกับข้อมูลสังเคราะห์และภาพหน้าจอจริงที่ผู้วิจัยจัดทำขึ้น วัด Word Accuracy ตามสูตร
$$“Word Accuracy” = 1 - “Word Error Rate”$$
 เนื่องจากผู้ใช้รับรู้ความผิดพลาดระดับคำมากกว่าระดับตัวอักษร การวัดร่วมกับ CER ช่วยให้เห็นภาพทั้งสองระดับ ทำให้ครอบคลุมมุมมองผู้ใช้ปลายทาง วัดเวลาประมวลผลเฉลี่ยต่อภาพบน CPU เนื่องจากในกรณีการทำงานออฟไลน์ CPU เป็นตัวกำหนดประสิทธิภาพหลัก ขั้นตอนการทดลอง ดำเนินการตามขั้นตอนต่อไป: ตั้งค่าสภาพแวดล้อมการทดสอบโดยกำหนดค่า \$“seed” = 42\$ เพื่อให้สามารถทำซ้ำได้ แล้วจึงเตรียมชุดข้อมูลทดสอบ โดยการดาวน์โหลดชุดข้อมูล ThaiOCRBench และ thai-ocr-evaluation (104 ภาพ) จากแหล่งต้นทาง เก็บใน ./data/ocr/test/ รันสคริปต์ประเมินผล OCR ด้วยคำสั่ง python scripts/eval_ocr.py –model PP-OCRV5-mobile –dataset thaiocrbench –output ./results/ocr.json –device cpu –seed 42 (เปลี่ยน –model PP-OCRV5-mobile เป็น –model ThaiTrOCR สำหรับโมเดล ThaiTrOCR) สคริปต์ทำงานอัตโนมัติตามลำดับ: (ก) อ่านภาพทีละภาพด้วย OpenCV (ข) ปรับขนาดภาพตามข้อกำหนดของแต่ละโมเดล สำหรับ PP-OCRV5 ใช้ 512×512 px และ ThaiTrOCR ใช้ 384×384 px โดยใช้การปรับขนาด แบบ bilinear interpolation (ค) รัน OCR inference บน CPU (ง) เปรียบเทียบข้อความส่งออกกับ ground truth ด้วยระยะทาง Levenshtein (จ) บันทึก CER, Word Accuracy และ latency (ms) ของแต่ละภาพลงไฟล์ JSON คำนวณค่า CER เฉลี่ยของทั้งชุดทดสอบ ทำซ้ำขั้นตอนที่ 3–5 ครบ 5 รอบ แล้วบันทึกค่า CER ของแต่ละรอบ แล้วจึงรายงานเป็นค่าเฉลี่ยเลขคณิตและส่วนเบี่ยงเบนมาตรฐาน เปลี่ยนโมเดลเป็น ThaiTrOCR และทำซ้ำขั้นตอนก่อนหน้า การแปลภาษา (MT) ค่า Bilingual Evaluation Understudy (BLEU) วัดค่า BLEU Score (Bilingual Evaluation Understudy) ซึ่งวัดความแม่นยำของการจับคู่ \$n\$-gram ระหว่างผลลัพธ์จากโมเดลกับข้อความอ้างอิงที่แปลโดยมนุษย์ ร่วมกับค่าปรับลดความสั้น (Brevity Penalty: BP) ตามสูตร
$$“BLEU” = “BP” \times \exp(\sum w_n \log p_n)$$
 โดย \$N = 4\$, \$p_n\$ คือความแม่นยำของ \$n\$-gram, และ \$“BP”\$ คือ brevity penalty ความเร็วในการแปล วัดจำนวน tokens ต่อวินาที บน CPU เพระบอกว่ามีความเร็วเพียงพอต่อ real-time หรือไม่ ขั้นตอนการทดลอง ดำเนินการตามขั้นตอนต่อไป: ตั้งค่าสภาพแวดล้อมการทดสอบโดยกำหนดค่า \$“seed” = 42\$ เพื่อให้สามารถทำซ้ำได้ แล้วจึงเตรียมชุดข้อมูลทดสอบ โดยการดาวน์โหลดชุดข้อมูล FLORES-200 ภาษาไทยและอังกฤษ เป็นจำนวน 3,001 คู่ประโยค แล้วเก็บไฟล์ไว้ใน ./data/mt/test/flores200.th และ ./data/mt/test/flores200.en รันสคริปต์ประเมินผล MT ด้วยคำสั่ง python scripts/eval_mt.py –model NLLB-200-distilled-600M –src th –tgt en –dataset flores200 –output ./results/mt.json –device cpu –beam 4 –max_len 256 –seed 42 โดยเปลี่ยน –model NLLB-200-distilled-600M เป็น –model NLLB-200-distilled-1.3B สำหรับโมเดลขนาด 1.3B สคริปต์ทำงานอัตโนมัติตามลำดับดังนี้: (ก) อ่านประโยคภาษาไทยจาก flores200.th ทีละประโยค (ข) tokenize ด้วย SentencePiece tokenizer ของ NLLB-200 (ค) แล้วทำการ MT inference บน CPU ด้วย

beam search ด้วยค่าพารามิเตอร์ดังนี้ \$“beam size” = 4\$, \$“max length” = 256\$ tokens, \$“length penalty” = 1.0\$ (ง) detokenize ข้อความที่แปลแล้ว (จ) เปรียบเทียบกับ reference จาก flores200.en เพื่อคำนวณ BLEU score (ฉ) บันทึก BLEU, ความเร็วในการแปล และ ความหวังของแต่ละประโยคลงไฟล์สกุล JSON คำนวณค่า BLEU โดยรวมจากชุดประเมิน FLORES-200 ทั้งหมด ทำซ้ำขั้นตอนที่ 3–5 ครบ 5 รอบ แล้วจึงรายงานเป็นค่าเฉลี่ยเลขคณิตและส่วนเบี่ยงเบนมาตรฐาน เปลี่ยนโมเดลเป็น NLLB-200-distilled-1.3B และทำซ้ำขั้นตอนที่ 3–6 เพื่อเปรียบเทียบประสิทธิภาพของระบบโดยรวม ระยะเวลาหน่วยแยกตามส่วนประกอบ วัดระยะเวลาหน่วยแยกตาม ส่วนประกอบของโปรแกรมด้วย performance.now() ใน frontend (JS) และ std::time ใน Rust backend รายงานเป็น ms เป็นการชี้จุดที่ทำให้ช้าลงได้ชัดเจนที่สุด และช่วยบอกว่าควรปรับปรุงส่วน ไหนก่อน RAM และ Disk Usage วัด RAM ในสถานะ idle (ไม่แปล) และ active (กำลังประมวลผล) ด้วยเครื่องมือ psutil วัดพื้นที่เก็บรวมทุกโมเดล runtime และแอป เพราะระบบต้องทำงานบนเครื่อง ทั่วไปที่อาจมี RAM จำกัด เวลาเปิดใช้งานครั้งแรก นอกจากนี้ ยังวัดเวลาตั้งแต่เปิดแอปจนพร้อมใช้งาน ครั้งแรก เพราะสำคัญในสถานการณ์ฉุกเฉินที่ต้องการความรวดเร็ว ขั้นตอนการทดลอง ดำเนินการตาม ขั้นตอนต่อไป: ติดตั้งเครื่องมือ Tauri CLI v2 (cargo install tauri-cli --version ^2) และโปรแกรม Rust 1.78.0 สำหรับ สร้างแอปพลิเคชัน GOaT แล้วสร้างโปรแกรม GOaT เวอร์ชันทดสอบ cargo tauri build --config src-tauri/tauri.conf.release.json จากนั้นติดตั้งไฟล์ .msi (Windows) หรือ .deb (Linux) จากกระบวนการสร้าง ก่อนเริ่มวัดผล ให้เปิดแอป GOaT 1 ครั้งเพื่อให้ระบบสร้าง cache และโหลดข้อมูล แล้วปิดแอป วัดเวลาเปิดใช้งานครั้งแรก: (ก) รีสตาร์ทเครื่อง OS รอจน เข้าสู่หน้าจอหลักและรออีก 30 วินาทีให้โปรแกรมพื้นหลังโหลดเสร็จ (ข) เริ่มจับเวลาที่จุดเริ่มต้นของ ฟังก์ชัน main() ใน Rust backend ด้วย std::time::Instant (ค) เปิดแอป GOaT โดย double-click ที่ไอคอนแอป (ง) หยุดจับเวลาเมื่อ frontend แสดง overlay และปุ่ม Capture พร้อมใช้งาน (จ) บันทึกเวลาในหน่วย ms (ฉ) ทำซ้ำขั้นตอน (ก)–(จ) ครบ 5 รอบ รายงานค่าเฉลี่ยเลขคณิตและ ส่วนเบี่ยงเบนมาตรฐาน วัดระยะเวลาหน่วยแยกตามส่วนประกอบ: (ก) เปิด GOaT แล้วจับภาพหน้าจอ (ข) ใช้ performance.now() ใน frontend JS จับเวลาตั้งแต่กด Capture จนได้รับผลลัพธ์กลับมา (ค) ใน Rust backend ใช้ std::time::Instant จับเวลาแยก 3 จุด ได้แก่ ocr_start ถึง ocr_end, mt_start ถึง mt_end, render_start ถึง render_end (ง) บันทึก timestamp ทุกจุดลงไฟล์ JSON (จ) ทำซ้ำกับภาพต่างกัน 100 ภาพ รายงานค่าเฉลี่ยเลขคณิตและส่วนเบี่ยงเบนมาตรฐาน ของแต่ละ ส่วนประกอบ และร้อยละของเวลาทั้งหมด วัด RAM Usage: (ก) รันสคริปต์ monitor_ram.py ซึ่ง เรียก psutil.Process().memory_info().rss ที่ sampling rate 1 Hz (ข) สภาวะ idle: เปิด GOaT ทิ้งไว้ 60 วินาทีโดยไม่กดปุ่มใดๆ บันทึก RAM ทุก 1 วินาที (ค) สภาวะ active: ส่งภาพ 1 ภาพผ่าน pipeline ครบ 1 cycle บันทึกค่า peak RAM ขณะประมวลผล (ง) รายงานค่าเฉลี่ยเลขคณิตและส่วน เบี่ยงเบนมาตรฐาน สำหรับ idle และ peak value สำหรับ active วัดพื้นที่เก็บข้อมูลที่แอปใช้ เพื่อ คำนวณขนาดรวมของไฟล์เดอร์แอป (รวมโมเดลและ dependencies ต่างๆ) การวิเคราะห์ทางสถิติ การทดสอบ Paired t-test เราเปรียบเทียบคู่โมเดล (ThaiTrOCR vs PP-OCrv5, NLLB-600M vs 1.3B, ก่อน vs หลัง fine-tune) ด้วย paired t-test ระดับนัยสำคัญ $\alpha = 0.05$ ใช้ paired test เพราะแต่ละคู่ทดสอบบนชุดข้อมูลชุดเดียวกัน การหาค่า Cohen’s d รายงาน Cohen’s d ตามสูตร $d = (\mu_1 - \mu_2) / \sigma_{\text{pooled}}$ โดย $\sigma_{\text{pooled}} = \sqrt{(\sigma_1^2 + \sigma_2^2) / 2}$

+ σ^2 / 2) \$ แปลผลตามเกณฑ์ของ Cohen: \$d \geq 0.2\$ \$ = \$ เล็ก, \$d \geq 0.5\$ \$ = \$ กลาง, \$d \geq 0.8\$ \$ = \$ ใหญ่ รายงาน 95% CI ของทุกเกณฑ์การวัดคำนวณจาก bootstrap 10,000 รอบ เพื่อให้ค่าประมาณความไม่แน่นอน การประเมินประสิทธิภาพเชิงคุณภาพ สภาพแวดล้อมในการประเมิน ใช้ผู้ประเมิน 3 คนที่เป็นเจ้าของภาษาไทยและคนที่พูดอังกฤษได้ เนื่องจากจำนวน 3 คนเป็นจำนวนขั้นต่ำที่ให้ผลการรายงานที่เชื่อถือได้ สุ่มลำดับ 300 ตัวอย่าง แบบปกปิดว่าเป็นของโปรแกรม GOaT หรือ baseline การให้คะแนน Direct Assessment วัดค่า Adequacy (0-100) เพื่อแปลผลว่าความหมายตรงกับต้นทางหรือไม่ และ Fluency (0-100) เพื่อแปลผลว่าภาษาไทยเป็นธรรมชาติหรือไม่ ทั้งสองอย่างนี้ครอบคลุมทั้งความถูกต้องเชิงความหมายและความเป็นธรรมชาติของภาษา การวิเคราะห์ประเภทความผิดพลาด (Error Categorization) แยกตามสาเหตุ 3 ประเภท: Type A สำหรับ OCR ผิดพลาดเท่านั้น Type B สำหรับ MT ผิดพลาดเท่านั้น และ Type C ผิดพลาดทั้งคู่ นับจำนวนและร้อยละของตัวอย่างในแต่ละประเภท ขั้นตอนการดำเนินการประเมิน ดำเนินการตามขั้นตอนต่อไปนี้: คัดเลือกผู้ประเมิน 3 คน ที่เป็นเจ้าของภาษาไทยและคนที่พูดอังกฤษได้ ไม่ใช่ผู้พัฒนา GOaT และไม่เคยเห็นระบบมาก่อน จัดเตรียมตัวอย่างทดสอบ 300 ตัวอย่าง โดยสุ่มจากภาพหน้าจอจริง 500 ภาพ จากนั้นสุ่มลำดับการนำเสนอแบบปกปิดว่าเป็นของโปรแกรม GOaT หรือ baseline จัดทำแบบประเมิน แต่ละตัวอย่างประกอบด้วย: (ก) ภาพหน้าจอต้นฉบับ (ข) ข้อความที่ระบบแปลได้ (ค) reference translation โดยผู้เชี่ยวชาญ (ง) ช่องคะแนน Adequacy 0–100 (เพื่อแปลผลว่าความหมายตรงกับต้นทางหรือไม่) (จ) ช่องคะแนน Fluency 0–100 (เพื่อแปลผลว่าภาษาไทยเป็นธรรมชาติหรือไม่) จัด calibration session ก่อนเริ่มประเมินจริง: ผู้ประเมินทั้ง 3 คนให้คะแนนตัวอย่าง 10 ตัวอย่างร่วมกัน อภิปรายความแตกต่างของคะแนนเพื่อปรับมาตรฐานเดียวกัน ใช้เวลา 30 นาที ผู้ประเมินแต่ละคนให้คะแนน 300 ตัวอย่างอิสระต่อกัน โดยไม่ปรึกษากัน ใช้เวลาไม่เกิน 2 ชั่วโมง พักทุก 50 ตัวอย่างเพื่อลดความเหนื่อยล้า รวบรวมคะแนน วิเคราะห์ inter-annotator agreement ด้วย Kendall's τ_b หาก $\tau_b < 0.5$ ให้จัด calibration session เพิ่มเติมและประเมินซ้ำ คัดแยกชนิดความผิดพลาด: (ก) นำตัวอย่างที่ \$“Adequacy” < 80\$ หรือ \$“Fluency” < 80\$ มาวิเคราะห์ (ข) ผู้ประเมินจำแนกสาเหตุตามเกณฑ์ Type A Type B หรือ Type C (ค) ผู้วิจัยตรวจทานซ้ำเพื่อยืนยันการจำแนก (ง) นับจำนวนและร้อยละของตัวอย่างในแต่ละประเภท รายงานผลโดยรวม การวิเคราะห์เปรียบเทียบ เทียบกับการทำงานแยกขั้นตอน เปรียบเทียบ GOaT กับการทำงานที่ใช้ OCR tool (PaddleOCR CLI) และ MT tool (TranslateLocally) แยกกันคนละโปรแกรม วัดเวลาตั้งแต่เริ่มจัดภาพหน้าจอจนได้ค่าแปล และวัดจำนวนขั้นตอนที่ผู้ใช้ต้องทำ ขั้นตอนการทดลองเปรียบเทียบ ดำเนินการตามขั้นตอนต่อไปนี้: ตั้งค่าการทดสอบเทียบกับการทำงานแยกขั้นตอน: เปิด PaddleOCR CLI ด้วยคำสั่ง `paddleocr -image input.png -lang th -use_angle_cls true -use_gpu false` จากนั้นคัดลอกข้อความที่ OCR ได้ แล้วจึงเปิด TranslateLocally จากนั้นจึงวางข้อความ แล้วกดปุ่ม Translate แล้วคัดลอกผลลัพธ์ การแปล จับเวลาตั้งแต่เริ่มเปิด PaddleOCR CLI จนได้ผลลัพธ์การแปล พร้อมนับจำนวนขั้นตอนทั้งหมดที่ผู้ใช้ต้องทำ บันทึกผล ทำซ้ำ 5 รอบกับภาพต่างกัน 5 ภาพ วัดเวลาการทำงานของ GOaT: เปิด GOaT แล้วกดปุ่ม Capture และอ่านผลจาก overlay จับเวลาตั้งแต่กด Capture จน overlay แสดงผลการแปลครบถ้วน บันทึกผล ทำซ้ำ 5 รอบกับภาพชุดเดียวกับข้อก่อนหน้า เปรียบเทียบเวลาเฉลี่ยและจำนวนขั้นตอนระหว่าง Separate-tool กับ GOaT ด้วยการทดสอบ paired t -test ($\alpha = 0.05$) แล้วรายงานเวลาที่ลดลงและขั้นตอนที่ลดลง